

QueryMaster: An End-to-End Text-to-SQL Autonomous Agent

Mehmet Zahit Avcı 22040102030

Abstract— The rapid growth of data requires democratized access to databases for non-technical users. Text-to-SQL systems bridge this gap by translating natural language into executable SQL queries. In this paper, we present QueryMaster, an autonomous end-to-end Text-to-SQL agent. The core inference engine utilizes a quantized Large Language Model (Qwen3-8B-4bit), fine-tuned via Low-Rank Adaptation (LoRA) optimized for Apple Silicon using the MLX framework. The model is trained on complex, cross-domain datasets (BIRD and SynSQL) and integrated into an interactive web application built with Streamlit. The system goes beyond simple text generation by providing a dynamic SQLite execution engine, automatic Entity-Relationship (ER) diagram generation, query execution flowcharts, and rigorous Execution Accuracy validation against Gold SQL queries. Our model achieves a strict Execution Accuracy of 67.4% on the Spider 1.0 development set, demonstrating strong generalization capabilities in complex, multi-hop relational schema queries.

Index Terms— Text-to-SQL, Large Language Models, Generative AI, Parameter-Efficient Fine-Tuning (LoRA), Database Management, Apple Silicon (MLX).

I. INTRODUCTION

Relational database management systems (RDBMS) are the backbone of modern data infrastructure. However, querying these databases requires proficiency in Structured Query Language (SQL), creating a barrier for non-technical stakeholders. Recent advancements in Large Language Models (LLMs) have shown promising results in translating natural language queries into SQL.

Despite this progress, deploying such models locally presents significant computational challenges. The objective of this project, QueryMaster, is to build a lightweight, highly accurate, and fully autonomous Text-to-SQL agent that can run efficiently on consumer-grade hardware (Apple Silicon). We aim not only to generate SQL but to provide a comprehensive analytical environment where users can upload databases, visualize schemas, and mathematically verify query results.

II. RELATED WORK / BACKGROUND

Historically, Text-to-SQL tasks were tackled using rule-based systems or early sequence-to-sequence models (e.g., LSTMs). With the advent of Transformers, models began treating the

problem as a translation task. Benchmarks like Spider introduced complex, cross-domain databases, pushing models to generalize better. More recently, the BIRD benchmark introduced massive, noisy, and real-world database schemas, establishing a new frontier in the field.

To deploy large models locally without massive GPU clusters, Parameter-Efficient Fine-Tuning (PEFT) techniques like LoRA (Low-Rank Adaptation) freeze the pre-trained model weights and inject trainable rank decomposition matrices. Furthermore, Apple's MLX framework provides native array processing on Apple Silicon, enabling efficient 4-bit quantization and fine-tuning using Unified Memory.

III. METHODOLOGY / MODEL AND DATASET

A. Datasets and Preprocessing

To ensure the model can handle diverse schemas, we curated a training dataset combining the BIRD and SynSQL databases. The data was preprocessed and formatted into the ChatML format, mapping user questions and database Data Definition Language (DDL) schemas to the target SQL queries.

B. Model Architecture and Fine-Tuning

The base model selected was Qwen3-8B, quantized to 4-bit precision to reduce memory footprint. We employed LoRA via the mlx-lm library to train the model locally. The prompt engineering was carefully designed to instruct the model to act as a "Senior SQLite Data Analyst," enforcing strict rules on Foreign Key matching and intermediate JOIN logic to prevent hallucination.

C. System Architecture

QueryMaster is composed of multiple interconnected modules:

Inference Engine: Handles real-time chat prompting and SQL generation.

DDL Parser: Uses regular expressions to extract tables, columns, Primary Keys, and Foreign Keys (including inline constraints) from SQL schema texts.

Visualizer: Translates the parsed schema into Mermaid.js syntax, generating dynamic ER Diagrams and Execution Flowcharts.

Execution Sandbox: An embedded SQLite engine that loads uploaded .db files or generates synthetic data to execute both the predicted SQL and the expected Gold SQL.

IV. EXPERIMENTAL SETUP / IMPLEMENTATION DETAILS

The application was developed using Python and Streamlit for the frontend UI. The fine-tuning process was executed on an Apple Silicon M-series chip utilizing the MLX framework, bypassing the need for CUDA-based NVIDIA GPUs.

The evaluation pipeline was integrated directly into the application. When a user provides a "Gold SQL", the system executes both the generated query and the gold query on the SQLite database. Instead of a simple string comparison (which is brittle due to formatting differences), the system performs a Multiset Row Comparison, verifying if the actual returned rows and columns logically match.

V. RESULTS AND DISCUSSION

A. Quantitative Results

The fine-tuned model was evaluated on the Spider 1.0 dev set. We achieved a strict Execution Accuracy of 67.4%. This metric proves that the generated queries retrieve the exact correct data from the database, effectively handling GROUP BY, nested subqueries, and complex multi-table JOIN operations.

B. Qualitative Analysis and Application Features

Beyond numerical accuracy, the QueryMaster interface successfully implements:

- Dynamic ER Diagrams:** Automatically resolving schema relationships and displaying interactive entity charts.
- Logical Execution Flow:** Deconstructing generated SQL into its execution order (e.g., FROM -> JOIN -> WHERE -> GROUP BY -> SELECT), greatly enhancing the explainability of the AI's thought process.
- Robust Error Handling:** The system captures SQLite compilation/execution errors in real-time, providing feedback to the user.

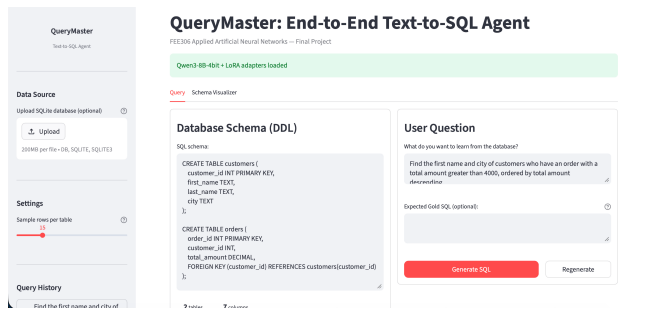


Fig. 1. Streamlit UI

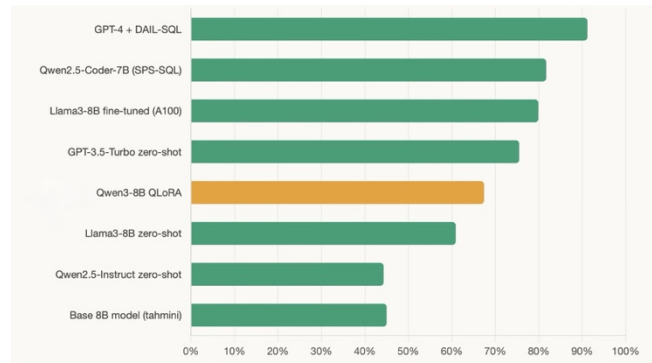


Fig. 2. Comparison with other models

VI. CONCLUSION AND FUTURE WORK

In conclusion, QueryMaster successfully demonstrates that a quantized, fine-tuned 8-billion parameter LLM can act as a highly accurate, autonomous database agent on consumer hardware. The project fulfills its objective by abstracting SQL complexity and providing visual and execution-based validation tools.

Future work could involve exposing the AI engine as a FastAPI microservice, developing a React frontend for broader deployment, and extending execution support to other dialects such as PostgreSQL and MySQL. Furthermore, integrating Agentic frameworks (like LangChain or LLM-based reflection) could allow the model to auto-correct its SQL based on execution errors.

REFERENCES

- [1] Yu, T., et al. "Spider: A Large-Scale Human-Labeled Dataset for Complex and Cross-Domain Semantic Parsing and Text-to-SQL Task." EMNLP (2018).
- [2] Li, J., et al. "BIRD: A Big Bench for Large-Scale Database Grounded Text-to-SQLs." NeurIPS (2023).
- [3] Hu, E. J., et al. "LoRA: Low-Rank Adaptation of Large Language Models." ICLR (2022).
- [4] Apple Machine Learning Research. "MLX: An array framework for Apple silicon." (2023).
- [5] Bai, J., et al. "Qwen Technical Report." arXiv preprint arXiv:2309.16609 (2023).